

# PWA INTERVIEW LEARNING GUIDE

## A Sequential Study Path: From Baseline Web Architectures to Offline-First Design

### 1. WHAT IS A WEB APPLICATION?

A **web application** is a client-server software system that is accessed over the internet via a web browser.

**Standard Request/Response Model:**

**Browser → Frontend (UI) → Internet → Backend (API) → Database**

In this model, the application depends entirely on the network. If the connection fails, requests fail, and the interface becomes completely unusable.

### 2. WHAT IS REACT?

**React** is a client-side JavaScript library used to build the user interface (UI) and manage component-based views. It handles DOM updates and application state on the browser.

**React = UI layer.** *React itself does not provide any offline database features, custom background processes, or installability characteristics. It only controls how views are rendered.*

### 3. WHAT IS A PWA?

**Progressive Web App (PWA)** — A web application that incorporates modern browser APIs to provide native app-like capabilities, including installability, offline access, caching, and background operations.

**React vs. PWA:** **React** is how we build the client-side UI. A **PWA** refers to the additional web layers (Manifest, Service Worker, client-side databases) configured to make that React UI run like an app.

**React UI / Components + PWA Features (Service Worker + Caching + Manifest) = React PWA**

### 4. WHY DO WE NEED PWAs?

**The Network Problem:** In a normal website, when the user is disconnected, any network request fails and the screen breaks. PWAs resolve this by keeping essential application files available locally.

**Important Caveat:** *A PWA does not automatically make the entire application work offline. Offline capability is not magic; it depends entirely on what resources and data the developer has designed and stored locally.*

### 5. HOW DOES A PWA ACTUALLY WORK?

A PWA sits on top of standard browser storage layers and introduces a network proxy. It intercepts outgoing fetch requests and determines whether to respond with local cached files or retrieve them from the server.

**Browser / UI ↓ PWA Layer ↓ Service Worker (Proxy) ↓ Cache / Local DB ↓ Internet (when available)**

### 6. WHAT IS A SERVICE WORKER?

A **Service Worker** is a background browser script running on a separate thread from the UI webpage. It acts as a client-side proxy between the application, the local cache, and the network.

**Website Requests File → Service Worker Intercepts → Check Local Cache → Serve Cached File OR Fetch from Network**

Because the Service Worker runs independently, it can process caching logic, handle requests, and run tasks even when the main page is closed.

### 7. WHAT IS CACHING?

**Caching** is the process of storing frequently needed assets locally so they can be loaded instantly or served during connection dropouts.

The PWA caches the application shell: HTML document, CSS files, JavaScript bundles, and static assets (icons, fonts).

**Internet Unavailable → Read Cached Application Shell Files → Application Shell Loads Offline**

**Key Concept:** *Caching static application files (HTML/CSS/JS) is separate from storing user database records (patient files, forms, or transactional data). Files are cached, but data is stored in a database.*

### 8. SERVICE WORKER vs INDEXEDDB

Do not confuse these two browser subsystems; they serve distinct functions:

- **Service Worker:** Handles network routing, proxies HTTP requests, and manages the file cache (HTML, CSS, JS).
- **IndexedDB:** A database that stores structured application data (JSON objects, client inputs, tables).

**Service Worker → Caches app assets / resources | IndexedDB → Stores structured user/application data**

## 9. WHAT IS INDEXEDDB?

**IndexedDB** is a client-side transactional database built into the browser for storing structured data locally.

**Offline Data Saving Flow:**

User fills form → **Offline?** → Store record in IndexedDB → Record remains available locally

**IndexedDB vs. localStorage:**

- **localStorage:** Synchronous (blocks UI), limited to 5MB, and only stores strings (requires JSON serialization).
- **IndexedDB:** Asynchronous (non-blocking), stores large/complex structured JSON datasets, and supports query indexes.

## 10. WHAT DOES OFFLINE-FIRST MEAN?

**Offline-first** is a design approach where the application is built to run entirely locally using local storage first, with server integration operating in the background when connection is available.

**Online:** App UI → Write/Read Backend API | **Offline:** App UI → Write/Read Local Storage (IndexedDB)

**Back Online:** Local Storage (IndexedDB) → Background Sync → Backend Database

**PWA vs. Offline-First:** PWA provides the browser capabilities (Service Worker, cache, IndexedDB APIs). **Offline-First** is the specific architectural design pattern used by developers to build the app's workflows. They are separate concepts.

## 11. HOW DOES SYNCHRONIZATION WORK?

**Synchronization** is the process of transmitting locally cached client-side data to the backend database when connection returns.

**Sequential Sync Flow:**

User enters data → IndexedDB → Internet returns → Sync Engine → REST API → Backend → Database

This flow ensures that client operations are never blocked by network state. The user continues working locally, and data is pushed when online.

## 12. DOES THE BACKEND NEED TO BE DIFFERENT?

**No, not fundamentally.** A PWA is a frontend architecture. The backend still exposes normal REST APIs and handles business logic, validation, and database storage. The frontend takes over the responsibility of queueing and syncing.

**PWA Frontend (Offline Queue & Sync) → REST API → Normal Backend Service → Database**

However, the backend should be designed to support synchronization safely, such as utilizing UUID keys and handling duplicate requests safely if client uploads are retried.

## 13. PWA vs NORMAL WEBSITE

- **Normal Website:** Browser → Network → Server. Requires a persistent connection. Outages block the UI immediately.
- **PWA:** Browser → Local resources when possible + Network when needed. Caches shell files to load offline.

PWAs provide installability, background sync, and offline support, but they do *not* mean that an internet connection is never needed.

## 14. PWA vs NATIVE MOBILE APP

- **PWA:** Runs in the browser using HTML, CSS, and JS. Distributed directly via URLs without app store approvals.
- **Native Mobile App:** Compiled code built for specific operating systems. Distributed via Google Play or Apple App Store.

**Summary:** PWAs leverage standard web technologies to achieve native-like app experiences.

## 15. PWA TECHNICAL INTERVIEW QUESTIONS & QUICK ANSWERS

• **Q: What is a PWA?**

**A:** A web app using modern browser APIs to support offline access, file caching, and mobile home screen installation.

• **Q: Is React a PWA?**

**A:** No. React is a UI library. PWA capabilities (manifest, service worker, databases) are added to the application separately.

• **Q: What is a Service Worker?**

**A:** A background browser thread serving as a network proxy. It intercepts requests to cache and serve static resources.

• **Q: What is caching?**

**A:** Keeping static application files locally on the device so they load instantly and can be served offline.

• **Q: What is IndexedDB?**

**A:** A browser database used to store large, structured application data asynchronously without blocking the UI.

• **Q: Service Worker vs IndexedDB?**

**A:** Service Worker handles network routing and asset caching; IndexedDB stores the structured data.

• **Q: What is offline-first?**

**A:** Designing the app to write and read locally first, using the internet as a background sync channel.

• **Q: Does PWA work without internet?**

**A:** Not automatically. Only features and pages explicitly cached or stored locally can work without connection.

• **Q: Does a PWA need a special backend?**

**A:** No. It uses standard REST/GraphQL APIs; the frontend manages offline queueing and background sync.

• **Q: What happens when internet returns?**

**A:** The client-side sync engine finds unsynced records in IndexedDB and posts them to the API.